

Giải Thích Chi Tiết Cách Hệ Thống Xử Lý Video Và Nhúng Tin

H.264/CAVLC Patchability-Aware Video Steganography

Tài liệu kỹ thuật cho dự án

Ngày 8 tháng 6 năm 2026

1 Câu Trả Lời Ngắn Gọn Trước

Nếu nói thật ngắn, hệ thống làm việc như sau:

Nó không sửa pixel của video. Nó sửa một vài dấu của các hệ số nén trong H.264, cụ thể là dấu của các trailing-one residual coefficients trong các block CAVLC của IDR/I-frame.

Một file H.264 được hệ thống nhìn như một cấu trúc nhiều tầng:

`file .h264 → NAL units → IDR slices → macroblocks → 4x4 residual blocks → coefficients`

Thông tin được nhúng vào tầng cuối cùng: **coefficient**. Nhưng không phải coefficient nào cũng được sửa. Hệ thống chỉ sửa các vị trí đã qua nhiều lớp lọc: an toàn về CAVLC, có thể vá lại bitstream, không làm chất lượng giảm quá mức, và đọc lại được sau khi tái tạo video.

2 Codec, H.264 Và Vì Sao Hệ Thống Không Sửa Pixel

2.1 Codec là gì?

Codec là bộ mã hóa/giải mã video. Tên này đến từ:

`coder + decoder = codec`

Video thô rất lớn. Codec như H.264 nén video thành bitstream nhỏ hơn để lưu, truyền, hoặc phát lại. Khi phát video, decoder đọc bitstream đó và dựng lại các frame.

2.2 Sửa pixel và sửa bitstream khác nhau thế nào?

Một hệ đơn giản có thể làm như sau:

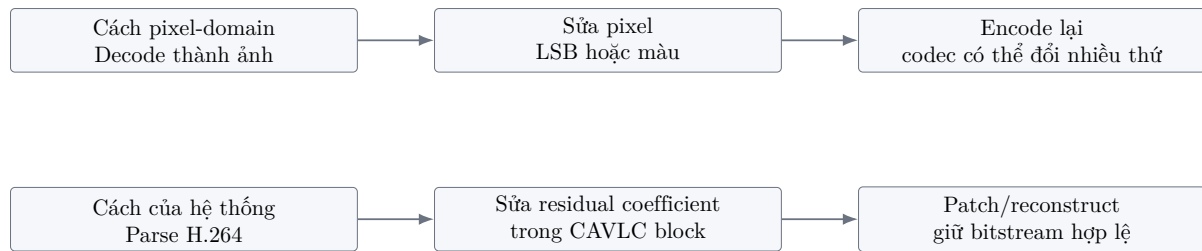
`decode video -> sửa pixel -> encode lại video`

Đó là cách làm ở **pixel domain**. Nó dễ hiểu nhưng có một vấn đề lớn: khi encode lại, codec có thể thay đổi rất nhiều thứ. Bit nhúng có thể mất, artifact có thể xuất hiện, và khó kiểm soát chính xác bitstream cuối cùng.

Hệ thống này đi theo hướng khác:

`parse H.264 bitstream -> sửa coefficient đã nén -> patch lại H.264`

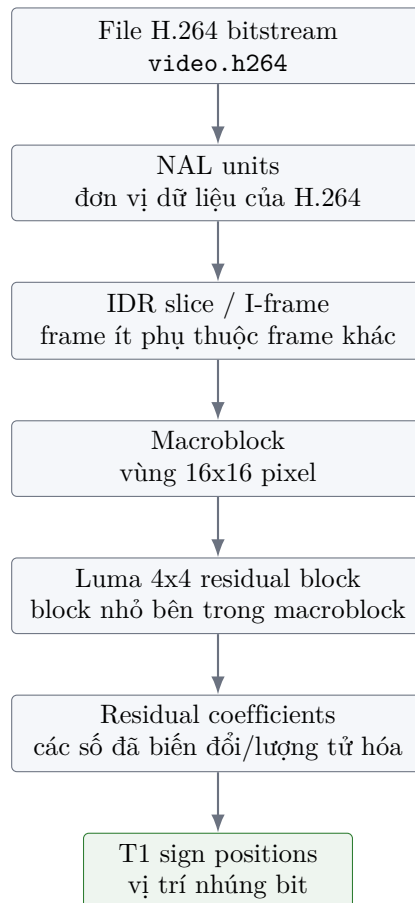
Đây là cách làm ở **compressed domain**. Nó gần codec hơn, vì hệ thống hiểu một phần cấu trúc H.264 và sửa đúng những chỗ mà codec đang dùng để biểu diễn residual.



Hình 1: Khác biệt giữa sửa pixel và sửa dữ liệu nén trong H.264.

3 File H.264 Được Hệ Thống Bóc Ra Như Thế Nào?

Phần này là lỗi để nhìn rõ hệ thống. Khi nhận một file `video.h264`, hệ thống không nhìn nó như một chuỗi byte vô nghĩa. Nó bóc file theo các tầng sau.



Hình 2: Tầng xử lý từ file H.264 đến vị trí coefficient có thể nhúng.

Một vị trí nhúng trong hệ thống có thể ghi gọn là:

`(macroblock_id, block_id, coefficient_id)`

Ví dụ:

`(496, 7, 1)`

Nghĩa là:

- macroblock số 496,
- block 4x4 số 7 trong macroblock đó,
- coefficient số 1 trong block đó.

4 Ví Dụ Đầy Đủ: Một Video H.264 Được Parse Ra Gì?

Phần này dùng một ví dụ minh họa gần với cách hệ thống làm việc. Đây không phải là dump đầy đủ của một file benchmark cụ thể, mà là một mẫu dữ liệu rút gọn để nhìn rõ pipeline. Trong code thật, các bước tương ứng nằm ở parser H.264, CAVLC parser, safety filter và bitstream reconstructor.

4.1 File H.264 ban đầu trông như thế nào?

Một file .h264 thô là một chuỗi byte. Trong H.264 Annex B, các NAL unit thường bắt đầu bằng start code:

00 00 00 01

Ví dụ một đoạn bitstream rút gọn có thể nhìn như sau:

Offset byte	Dữ liệu minh họa
0x00000000	00 00 00 01 67 ...
0x0000001A	00 00 00 01 68 ...
0x00000024	00 00 00 01 65 B8 21 ...
0x00004F10	00 00 00 01 65 88 40 ...

Hệ thống đọc các start code này để tách file thành các NAL unit. Sau đó nó đọc header của mỗi NAL để biết NAL đó là SPS, PPS, IDR slice hay loại khác.

4.2 Kết quả parse NAL mẫu

Sau bước parse NAL, hệ thống có thể có một bảng nội bộ giống như sau:

NAL	Offset	Type	Vai trò	Hệ thống làm gì?
0	0x00000000	7	SPS	Đọc kích thước, profile, thông tin frame.
1	0x0000001A	8	PPS	Đọc entropy mode, CAVLC/CABAC, tham số slice.
2	0x00000024	5	IDR slice	Có thể đi vào macroblock và tìm vị trí nhúng.
3	0x00004F10	5	IDR slice	Tiếp tục tìm block/coefficient dùng được.

Với baseline hiện tại, hệ thống quan tâm nhất tới NAL type 5, tức IDR slice. Các NAL SPS/PPS cần thiết để hiểu stream, nhưng không phải nơi nhúng payload.

4.3 Từ IDR slice đến macroblock

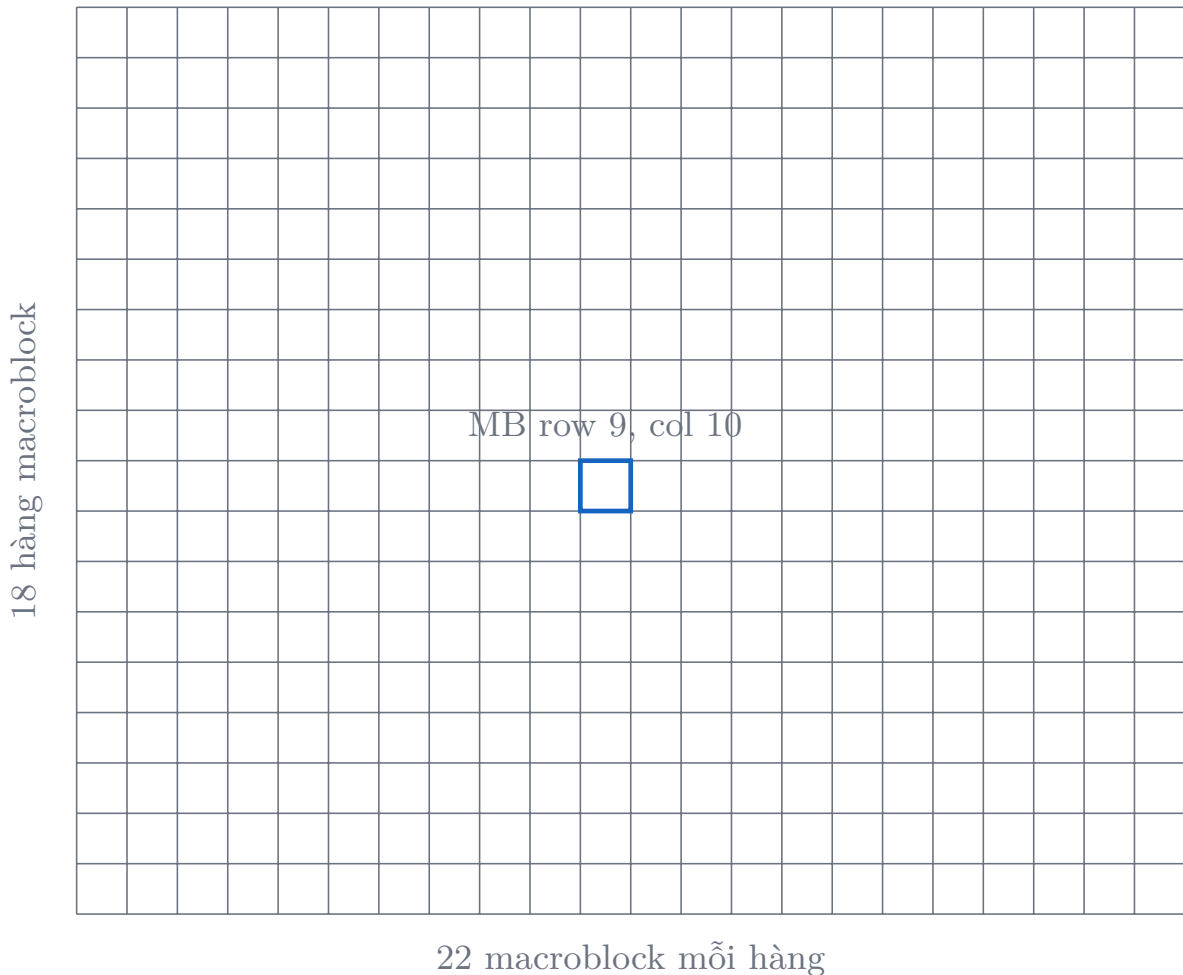
Một IDR slice chứa nhiều macroblock. Với video CIF 352x288, mỗi frame có:

22 macroblock theo chiều ngang $\times$ 18 macroblock theo chiều dọc = 396 macroblock/frame.

Một bảng macroblock rút gọn có thể như sau:

Frame	Macroblock	Hàng/cột MB	Slice-local MB	Ghi chú
12	4752	row 0, col 0	0	IDR frame bắt đầu.
12	4753	row 0, col 1	1	Có luma residual blocks.
12	4960	row 9, col 10	208	Có nhiều T1 candidates.
12	5147	row 17, col 21	395	Macroblock cuối frame.

Trong code, chỉ số macroblock thường được dùng theo dạng global index để các frame khác nhau không bị trùng vị trí.



Hình 3: Một frame CIF 352x288 được chia thành lưới 22x18 macroblock. Ô được tô là một macroblock minh họa.

4.4 Từ macroblock đến các block 4x4

Mỗi macroblock luma có thể được nhìn như 16 block 4x4. Hệ thống đánh số các block con này và đọc residual coefficients trong từng block.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Hình 4: Một macroblock luma có thể được chia thành 16 block 4x4, đánh số 0 đến 15.

Ví dụ, nếu hệ thống nói vị trí:

(4960, 7, 13)

thì có thể hiểu là:

- macroblock global index 4960,
- block 4x4 số 7,
- coefficient index 13 trong block đó.

4.5 Một block 4x4 sau khi parse trông như thế nào?

Giả sử block 7 trong macroblock 4960 có coefficients như sau:

0	5	-4	0
6	-1	0	0
8	0	-1	1
0	0	0	-1

Hệ thống cũng lưu thêm metadata CAVLC cho block này:

Trường nội bộ	Giá trị minh họa
<code>macroblock_id</code>	4960
<code>block_id</code>	7
<code>nC</code>	3, ngữ cảnh láng giềng dùng khi decode/encode CAVLC.
<code>bit_offset</code>	18422, vị trí bắt đầu block trong RBSP/slice.
<code>bit_length</code>	21, độ dài mã CAVLC ban đầu của block.
<code>trailing_ones</code>	2, số T1 sign flags thật mà block đang có.
<code>t1_positions</code>	[13, 15], coefficient index có thể là T1 sign positions.

Sau bước này, hệ thống chưa nhúng gì cả. Nó mới chỉ biết: “block này có vài coefficient có vẻ đáng xem xét”.

4.6 Bảng candidate positions mẫu

Từ nhiều macroblock và block, safety filter tạo ra danh sách candidate:

Candidate	Frame	MB	Block	Coeff	Trạng thái ban đầu
0	12	4960	7	13	T1 sign candidate.
1	12	4960	7	15	T1 sign candidate nhưng cùng block với candidate 0.
2	13	5356	2	11	T1 sign candidate.
3	14	5752	9	14	T1 sign candidate.

Sau đó per-block limit có thể giữ candidate 0 và loại candidate 1, vì cả hai nằm trong cùng (MB, block). Đây là lý do số candidate giảm trước khi thành operating positions.

4.7 Một bước patchability check mẫu

Với candidate (4960, 7, 13), hệ thống thử sửa dấu coefficient rồi re-encode block bằng CAVLC:

Kiểm tra	Kết quả minh họa	Ý nghĩa
Decode block gốc	pass	Parser đọc được block với nC đúng.
Encode lại block gốc	21 bit	Khớp độ dài ban đầu.
Sửa coefficient	$-1 \rightarrow +1$	Nhúng bit 1.
Encode block mới	21 bit	Độ dài không đổi, có thể patch.
Forward round-trip	pass	Decode block mới vẫn ra coefficient mong muốn.
Ancestor/retro check	pass	Không làm block trước nuốt nhầm bit của block này.

Nếu mọi dòng đều pass, candidate này mới được xem là patchable. Nếu chỉ một dòng fail, hệ thống loại block hoặc vị trí đó khỏi capacity thật.

4.8 Từ candidate đến operating position

Sau nhiều bước lọc, danh sách có thể rút lại như sau:

Tầng	Số vị trí minh họa	Candidate 0	Candidate 1	Candidate 2
Raw safe	433,256	giữ	giữ	giữ
Patchable usable	1,391	giữ	loại	giữ
Validated pool	1,316	giữ	loại	giữ
Operating positions	1,232	giữ	loại	giữ

Khi tới operating positions, mỗi vị trí đã đủ điều kiện để mang một bit payload trong benchmark-grade path.

5 Residual Coefficient Là Gì?

H.264 thường không lưu trực tiếp từng pixel. Nó dự đoán nội dung của một block, rồi lưu phần sai khác giữa dự đoán và dữ liệu thật. Phần sai khác này gọi là **residual**. Sau biến đổi và lượng tử hóa, residual trở thành một dãy số gọi là **residual coefficients**.

Ví dụ một block 4x4 có thể có 16 coefficient:

0	5	-4	0
6	-1	0	0
8	0	-1	1
0	0	0	-1

Hệ thống không sửa tất cả các số này. Nó quan tâm đặc biệt tới các coefficient ± 1 ở cuối dãy non-zero của CAVLC, vì các coefficient này có thể được mã hóa như **trailing-one sign flags**.

6 CAVLC Và Trailing Ones

6.1 CAVLC là gì?

CAVLC là viết tắt của:

Context-Adaptive Variable Length Coding

Đây là cách H.264 baseline mã hóa residual coefficients. Điểm quan trọng là **variable length**: giá trị khác nhau có thể tạo ra chuỗi bit dài ngắn khác nhau.

Ví dụ minh họa:

Giá trị coefficient	Mã minh họa	Độ dài
1	1	1 bit
4	001011	6 bit
5	0001110	7 bit

Bảng trên chỉ minh họa ý tưởng, không phải bảng mã H.264 đầy đủ. Ý chính là: nếu sửa coefficient bừa bãi, độ dài CAVLC có thể đổi. Khi độ dài đổi, việc vá trở lại bitstream trở nên nguy hiểm.

6.2 Trailing ones là gì?

Trong CAVLC, tối đa ba coefficient cuối có trị tuyệt đối bằng 1 có thể được mã hóa đặc biệt. Chúng được gọi là **trailing ones**, hay viết tắt là **T1**. Với T1, phần dấu (+/-) được mã hóa bằng các bit dấu riêng.

Điều này rất hợp với nhúng tin:

Bit cần nhúng	Hành động trực quan	Coefficient sau cùng
0	đặt dấu âm	-1
1	đặt dấu dương	+1

Nếu coefficient thật sự là T1 sign flag, đổi dấu thường là thay đổi rất nhỏ và có thể giữ nguyên độ dài mã. Vì vậy hệ thống tập trung vào **T1 sign embedding**.

7 Ví Dụ Cụ Thể: Nhúng 4 Bit Vào Một Số Vị Trí

Giả sử payload có 4 bit đầu:

1 0 1 1

Hệ thống đã chọn được 4 vị trí T1 an toàn:

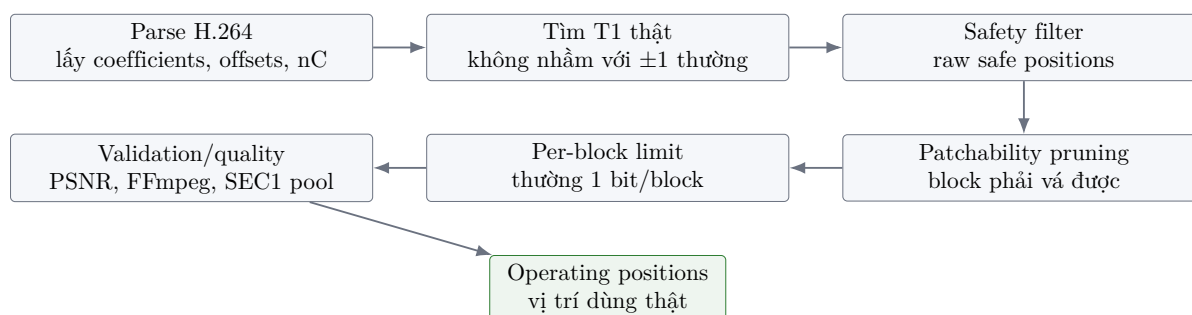
Bit	Vị trí	Coefficient trước	Coefficient sau
1	(12, 496, 1)	-1	+1
0	(13, 892, 1)	+1	-1
1	(14, 1288, 1)	+1	+1
1	(15, 1684, 1)	-1	+1

Điều quan trọng là mỗi dòng trong bảng không chỉ là “sửa một số”. Mỗi sửa đổi phải thỏa mãn:

- coefficient đúng là vị trí T1 sign flag;
- block sau khi sửa vẫn mã hóa được bằng CAVLC;
- độ dài bit của block không phá cấu trúc NAL/slice;
- số sửa đổi trong một block không vượt giới hạn;
- video sau tái tạo vẫn decode được;
- bit có thể đọc lại từ stego video.

8 Pipeline Chọn Vị Trí Nhúng

Đây là phần quan trọng nhất của hệ thống. Một vị trí chỉ được dùng nếu đi qua nhiều cửa kiểm tra.



Hình 5: Pipeline chọn vị trí nhúng từ H.264 bitstream đến operating positions.

8.1 Safety filter

Safety filter là lớp lọc đầu tiên. Nó trả lời câu hỏi:

“Vị trí này có vẻ an toàn về mặt cấu trúc CAVLC không?”

Nó kiểm tra các yếu tố như:

- vị trí có phải trailing-one sign flag thật không;

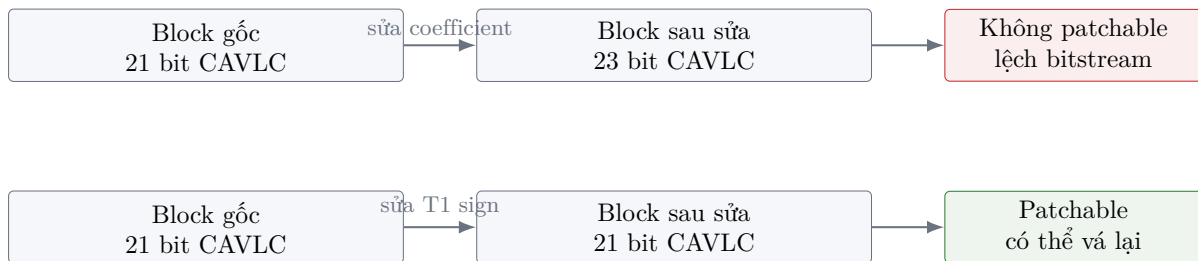
- sửa dấu có giữ được kiểu mã hóa mong muốn không;
- block không nằm trong nhóm đã biết là không patchable;
- thay đổi không làm zero/non-zero pattern nguy hiểm;
- magnitude và số sửa đổi không vượt ngưỡng.

8.2 Patchability-aware embedding

Patchability-aware embedding nghĩa là:

“Chỉ nhúng vào những vị trí mà sau khi sửa, block vẫn có thể được vá lại vào bitstream H.264 hợp lệ.”

Đây là một khái niệm rất quan trọng. Trong H.264/CAVLC, sửa một coefficient có thể làm block sau khi mã hóa dài hơn hoặc ngắn hơn. Nếu độ dài thay đổi, phần bit phía sau trong slice có thể bị lệch. Khi đó video có thể hỏng, parser có thể đọc sai, hoặc verifier không đọc lại được payload.



Hình 6: Patchability: sửa xong phải vá được vào đúng vị trí trong bitstream.

Ảnh hưởng của patchability:

- **Tăng độ đúng:** video sau cùng ít bị hỏng bitstream.
- **Tăng khả năng đọc lại:** bit đã nhúng có khả năng sống qua reconstruction.
- **Giảm capacity:** nhiều vị trí nhìn có vẻ dùng được sẽ bị loại.
- **Làm claim paper chặt hơn:** capacity được báo gần với khả năng dùng thật.

8.3 Per-block limit

Hệ thống thường giới hạn số bit nhúng trong mỗi block, ví dụ:

$$\text{max_modifications_per_block} = 1$$

Nghĩa là một block 4x4 dù có nhiều vị trí có vẻ dùng được, hệ thống cũng chỉ chọn tối đa một vị trí. Điều này giúp giảm méo cục bộ và làm reconstruction ổn định hơn. Đổi lại, capacity giảm.

8.4 Quality guard

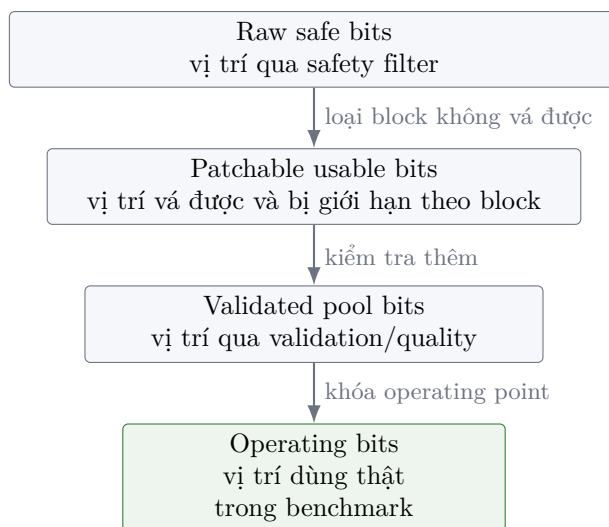
Quality guard đảm bảo video sau nhúng không bị xấu quá mức. Hệ thống dùng các metric như PSNR/SSIM, trong đó ngưỡng quan trọng của benchmark hiện tại là:

$$\text{min_modified_frame_psnr} \geq 40 \text{ dB}$$

Nếu một cấu hình nhúng làm frame tụt dưới ngưỡng, cấu hình đó không được coi là operating point mạnh.

9 Capacity: Vì Sao Không Thể Chỉ Đếm Candidate?

Một điểm mạnh của hệ thống là nó không nói “video có bao nhiêu coefficient thì có bấy nhiêu bit”. Nó chia capacity thành nhiều lớp.



Hình 7: Capacity funnel: càng xuống dưới càng gần capacity thật.

Ví dụ minh họa:

Lớp capacity	Số lượng ví dụ	Ý nghĩa
Raw safe bits	433,256	Vị trí có vẻ an toàn về cấu trúc CAVLC.
Patchable usable bits	1,391	Vị trí còn lại sau khi xét patchability và giới hạn theo block.
Validated pool bits	1,316	Vị trí đã qua validation/quality.
Operating bits	1,232	Vị trí thật sự dùng ở operating point.

Điểm quan trọng là: **raw safe bits không phải capacity thật**. Capacity thật hơn nằm ở patchable/validated/operating layers.

10 Payload Được Chuẩn Bị Như Thế Nào?

Về mặt video, hệ thống chỉ cần một chuỗi bit để nhúng. Chuỗi bit này đến từ payload blob:

`[4B message length][message][129B Groth16 proof]`

Ví dụ message benchmark:

`b"ZK-bench-v1.0!"`

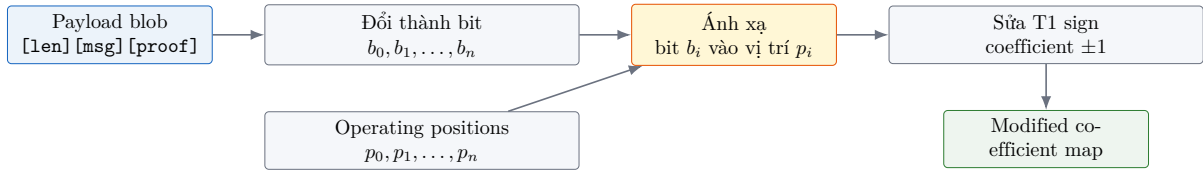
Nếu message dài 13 byte và proof dài 129 byte, payload chưa chaos là:

$$4 + 13 + 129 = 146 \text{ bytes} = 1168 \text{ bits}$$

Trong một số benchmark có chaos transform, payload operating có thể mở rộng lên khoảng 1232 bit.

11 Payload Được Nhúng Vào Video Như Thế Nào?

Sau khi có payload bits và operating positions, hệ thống ghép từng bit với từng vị trí:



Hình 8: Mỗi payload bit được đưa vào một vị trí coefficient đã chọn.

11.1 Ví dụ ánh xạ payload bits vào positions

Giả sử payload sau khi pack được đổi thành các bit đầu tiên:

1 0 1 1 0 0 1 0

Và operating positions đầu tiên là:

Bit index	Payload bit	Frame	MB	Block	Coeff
0	1	12	4960	7	13
1	0	13	5356	2	11
2	1	14	5752	9	14
3	1	15	6148	1	15
4	0	16	6544	6	12
5	0	17	6940	5	13
6	1	18	7336	8	15
7	0	19	7732	3	11

Sau đó hệ thống đi từng dòng:

Bit	Frame	MB	Block	Coeff	Trước	Sau
1	12	4960	7	13	-1	+1
0	13	5356	2	11	+1	-1
1	14	5752	9	14	+1	+1
1	15	6148	1	15	-1	+1
0	16	6544	6	12	-1	-1
0	17	6940	5	13	+1	-1
1	18	7336	8	15	-1	+1
0	19	7732	3	11	+1	-1

Bảng này là hình ảnh trực quan nhất của quá trình nhúng: payload bit được biến thành dấu của một coefficient T1 ở một block cụ thể.

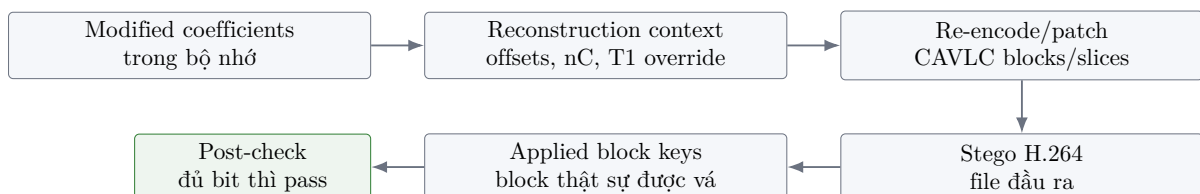
Ở mức một block, có thể hình dung như sau:

Block trước nhúng	Hành động	Block sau nhúng																																
<table><tr><td>0</td><td>5</td><td>-4</td><td>0</td></tr><tr><td>6</td><td>-1</td><td>0</td><td>0</td></tr><tr><td>8</td><td>0</td><td>-1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>-1</td></tr></table>	0	5	-4	0	6	-1	0	0	8	0	-1	1	0	0	0	-1	Payload bit = 1 chọn T1 sign position đặt coefficient thành +1	<table><tr><td>0</td><td>5</td><td>-4</td><td>0</td></tr><tr><td>6</td><td>-1</td><td>0</td><td>0</td></tr><tr><td>8</td><td>0</td><td>-1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>+1</td></tr></table>	0	5	-4	0	6	-1	0	0	8	0	-1	1	0	0	0	+1
0	5	-4	0																															
6	-1	0	0																															
8	0	-1	1																															
0	0	0	-1																															
0	5	-4	0																															
6	-1	0	0																															
8	0	-1	1																															
0	0	0	+1																															

Hình 9: Ví dụ trực quan: nhúng một bit bằng cách đổi dấu một coefficient T1.

12 Reconstruction: Làm Sao Video Sau Nhúng Vẫn Là H.264 Hợp Lệ?

Sau khi sửa coefficient trong bộ nhớ, hệ thống phải tạo lại file stego H.264. Đây là bước reconstruction.



Hình 10: Reconstruction biến coefficient đã sửa thành stego H.264 hợp lệ.

Sau reconstruction, hệ thống không tin mù rằng mọi bit đều đã được áp dụng. Nó kiểm tra lại:

- `requested_position_bits`: số vị trí dự định dùng;
- `applied_position_bits`: số vị trí thật sự được vá vào bitstream;
- `bits_required`: số bit payload cần nhúng.

Nếu `applied_position_bits < bits_required`, hệ thống báo fail ở `post_reconstruct_application`. Đây là một điểm rất quan trọng: hệ thống đo capacity sau khi bitstream đã được tái tạo, không chỉ trước khi tái tạo.

12.1 Ví dụ artifact sau khi nhúng

Sau một lượt nhúng thành công, thư mục output thường có các file dạng:

File	Ý nghĩa
<code>stego.h264</code>	Video H.264 đã chứa payload.
<code>stego.h264.positions.json</code>	Danh sách positions đã dùng để nhúng bit.
<code>stego.h264.meta.json</code>	Số bit required/embedded, capacity diagnostics, chaos flag.
<code>stego.h264.manifest.json</code>	Manifest có schema version, payload metadata, video hash, proof metadata.

Một `positions.json` rút gọn có thể nhìn như sau:

Index	Macroblock	Block	Coefficient
0	4960	7	13
1	5356	2	11
2	5752	9	14
3	6148	1	15

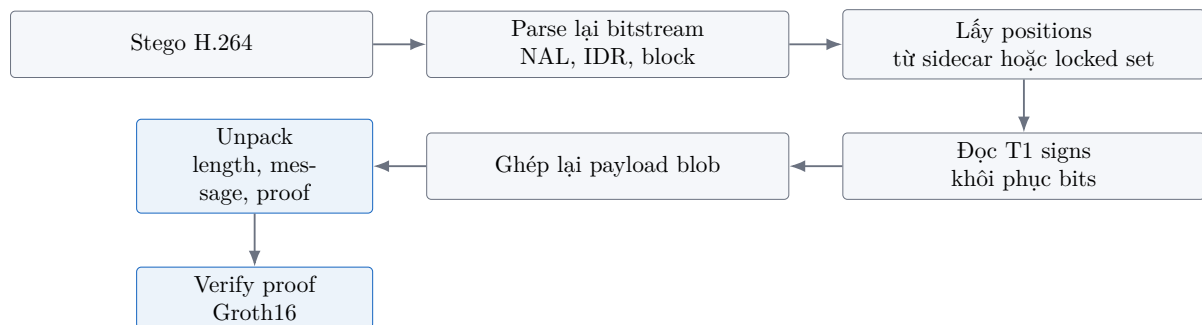
Một `meta.json` rút gọn có thể chứa:

Trường	Giá trị minh họa
<code>bits_required</code>	1232
<code>bits_embedded</code>	1232
<code>raw_safe_bits</code>	433256
<code>patchable_usable_bits</code>	1391
<code>requested_position_bits</code>	1232
<code>applied_position_bits</code>	1232

Nếu `applied_position_bits` nhỏ hơn `bits_required`, điều đó có nghĩa là một phần positions đã không sống qua reconstruction, và output đó không được xem là operating-point success.

13 Trích Xuất Payload Diễn Ra Như Thế Nào?

Trích xuất là pipeline ngược của nhúng.



Hình 11: Pipeline trích xuất: đọc dấu T1 ở đúng positions để dựng lại payload.

13.1 Ví dụ đọc ngược từ stego video

Giả sử verifier có `positions.json` với các dòng đầu:

Index	Macroblock	Block	Coefficient
0	4960	7	13
1	5356	2	11
2	5752	9	14
3	6148	1	15

Verifier parse `stego.h264`, đi tới đúng các vị trí đó và đọc dấu của coefficient:

Index	MB	Block	Coeff	Coefficient đọc được	Bit khôi phục
0	4960	7	13	+1	1
1	5356	2	11	-1	0
2	5752	9	14	+1	1
3	6148	1	15	+1	1

Các bit này được ghép lại thành payload blob:

1011...

Sau khi đủ số bit, hệ thống đọc 4 byte đầu để biết độ dài message, lấy phần message, lấy phần proof, rồi gọi verifier ZKP. Nếu một vài bit đầu header sai, toàn bộ quá trình unpack có thể hỏng vì độ dài message bị đọc sai. Đây là lý do các thử nghiệm blind-core sau này phải quan tâm rất nhiều tới header reliability.

Với verifier hiện tại:

- **Strict non-blind:** cần cover/original hoặc thông tin vị trí tương đương.
- **Sidecar-assisted near-blind:** không cần cover gốc nhưng cần sidecar như manifest/positions.
- **Blind-core:** hướng nghiên cứu, chưa phải claim chính của baseline hiện tại.

14 Cơ Chế ZKP: Proof Được Sinh Từ Đâu Và Sinh Như Thế Nào?

ZKP không chọn vị trí nhúng trong video. ZKP tạo ra một proof mật mã để chứng minh rằng message được nhúng có liên hệ đúng với một secret key, nhưng không tiết lộ secret key đó. Sau đó proof được nén thành bytes và đưa cho video embedding core giấu vào H.264 như dữ liệu bình thường.

14.1 Mệnh đề mà proof chứng minh

Circuit chính nằm trong:

`circuits/payload_verify.circom`

Circuit tên là PayloadVerify. Nó chứng minh quan hệ:

`commitment = SHA256(payload_hash || secret)`

Trong đó:

`payload_hash = SHA256(message)`

Nói bằng lời:

Người tạo proof biết một secret key sao cho khi ghép secret key đó với hash của message, rồi băm SHA256, ta nhận được commitment công khai.

Điểm quan trọng là secret key không được nhúng vào video và không xuất hiện trong public signals. Secret key chỉ đi vào witness khi tạo proof.

14.2 Public input và private input

Circuit có các input sau:

Loại input	Tên	Ý nghĩa
Public	<code>payload_hash[256]</code>	256 bit của SHA256(message).
Public	<code>commitment[256]</code>	256 bit commitment mong đợi.
Public	<code>payload_length</code>	Độ dài message theo byte.
Private	<code>secret[256]</code>	256 bit secret key, tức 32 byte.

Public input là phần verifier được phép biết. Private input là phần prover biết nhưng không lộ ra. Đây là phần “zero-knowledge” của hệ thống.

14.3 Ví dụ dữ liệu trước khi tạo proof

Giả sử message là:

```
b"ZK-bench-v1.0!"
```

và secret key là 32 byte:

```
00 01 02 03 ... 1f
```

Python bridge trong `src/zk_proof.py` tạo dữ liệu circuit như sau:

Bước	Giá trị tạo ra
1	<code>payload_hash = SHA256(message)</code>
2	<code>commitment = SHA256(payload_hash secret_key)</code>
3	Chuyển <code>payload_hash</code> thành 256 bit.
4	Chuyển <code>commitment</code> thành 256 bit.
5	Chuyển <code>secret_key</code> thành 256 bit private input.
6	Ghi thêm <code>payload_length = len(message)</code> .

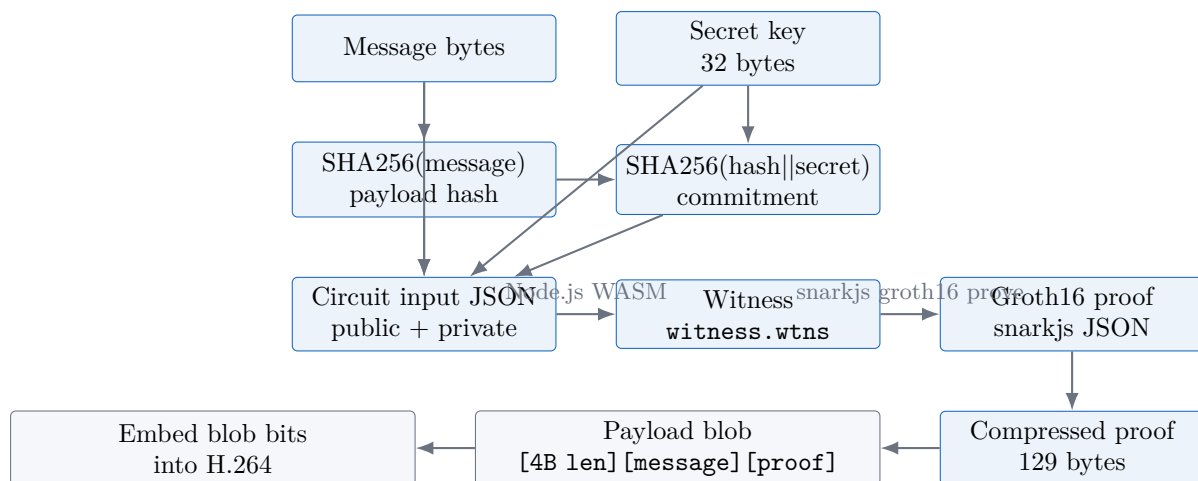
Trong code, logic này tương ứng với hàm `_build_circuit_input(payload_bytes, secret_key)`.

14.4 Workflow sinh proof

Hệ thống dùng `snarkjs` và `Groth16`. Các artifact quan trọng nằm trong `circuits/build/`:

Artifact	Vai trò
<code>payload_verify.wasm</code>	Chạy circuit để tạo witness từ input.
<code>generate_witness.js</code>	Script Node.js sinh witness.
<code>proving_key.zkey</code>	Proving key dùng để tạo Groth16 proof.
<code>verification_key.json</code>	Verification key dùng để verify proof.

Pipeline tạo proof:



Hình 12: Workflow tạo proof: message và secret tạo witness, witness tạo Groth16 proof, proof được nén rồi nhúng vào video.

14.5 Witness là gì?

Witness là tập giá trị làm cho circuit thỏa mãn. Trong hệ thống này, witness bao gồm secret key và các giá trị trung gian khi circuit tính:

$$\text{SHA256}(\text{payload_hash} \parallel \text{secret})$$

Nếu secret không đúng, circuit không thể tạo witness hợp lệ cho commitment đã công khai. Nếu witness không hợp lệ, Groth16 proof sẽ không verify.

Trong code, witness được tạo bằng:

```
node generate_witness.js payload_verify.wasm input.json witness.wtns
```

Sau khi tạo proof, các file tạm chứa secret/witness/proof/public được xóa. Điểm này quan trọng vì input JSON có chứa secret key ở dạng bit.

14.6 Proof JSON và proof bytes khác nhau thế nào?

snarkjs tạo proof ở dạng JSON, thường có các phần:

Trường proof	Ý nghĩa
pi_a	Điểm elliptic curve trong group G1.
pi_b	Điểm elliptic curve trong group G2.
pi_c	Điểm elliptic curve trong group G1.
protocol	groth16.
curve	bn128.

JSON này quá dài để nhúng trực tiếp. Vì vậy hệ thống nén proof thành 129 byte:

Thành phần	Kích thước
pi_a.x	32 byte
pi_b.x[0]	32 byte
pi_b.x[1]	32 byte
pi_c.x	32 byte
sign flags cho các tọa độ y	1 byte
Tổng	129 byte

Hàm tương ứng là:

```
proof_to_bytes(proof_dict)
```

Khi verify, hệ thống gọi:

```
bytes_to_proof(proof_bytes)
```

để dựng lại proof JSON từ 129 byte này.

14.7 Payload blob cuối cùng được nhúng vào video

Sau khi có proof bytes, hệ thống pack message và proof thành một blob:

```
[4 bytes message length][message bytes][129 bytes proof]
```

Ví dụ:

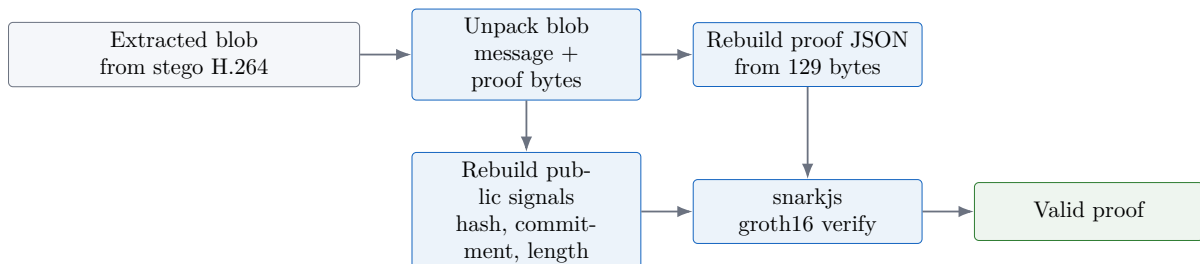
Thành phần	Kích thước	Ghi chú
Message length header	4 byte	Big-endian integer.
Message	13 byte	Ví dụ ZK-bench-v1.0! .
Compressed proof	129 byte	Groth16 BN128 proof.
Tổng	146 byte	1168 bit trước chaos.

Đây là dữ liệu mà video embedding core thật sự nhìn thấy. Từ góc nhìn của phần nhúng video, ZKP proof chỉ là 129 byte cần được giấu và đọc lại chính xác.

14.8 Verify sau khi extract diễn ra thế nào?

Sau khi verifier đọc lại payload blob từ stego video, hệ thống làm ngược lại:

1. Đọc 4 byte đầu để biết độ dài message.
2. Tách message bytes và 129 proof bytes.
3. Dựng lại proof JSON bằng `bytes_to_proof()`.
4. Tính lại `payload_hash = SHA256(message)`.
5. Tính lại `commitment = SHA256(payload_hash || secret_key)`.
6. Tạo public signals gồm `payload_hash`, `commitment`, `payload_length`.
7. Gọi `snarkjs groth16 verify` với `verification_key.json`.



Hình 13: Workflow verify: sau khi extract đúng bytes, hệ thống dựng lại proof và public signals rồi gọi Groth16 verifier.

14.9 ZKP và video embedding chia trách nhiệm thế nào?

Hai phần này có vai trò khác nhau:

Thành phần	Trách nhiệm
ZKP layer	Chứng minh message có commitment hợp lệ với secret key mà không lộ secret key.
Video embedding core	Giấu chính xác payload bytes vào H.264 và đọc lại đúng payload bytes đó.
Verifier	Kiểm tra rằng bytes đọc lại từ video tạo thành proof hợp lệ.

Nói cách khác:

ZKP trả lời: payload có quan hệ mật mã hợp lệ không?

Video embedding trả lời: payload có được giấu và đọc lại đúng không?

15 Workflow Thực Tế Khi Gọi `embed()`

Khi gọi public API `embed()`, pipeline thực tế đi qua các bước sau:

1. Kiểm tra input: video tồn tại, `circuits_dir` tồn tại, message không rỗng, secret key dài 32 byte.
2. Phân tích stream profile: all-intra hay inter-coded. Baseline mạnh nhất hiện tại là all-intra H.264/CAVLC.
3. Tạo Groth16 proof cho message và secret key.
4. Pack payload thành `[4B length] [message] [proof]`.
5. Nếu bật chaos, scramble payload bits và shuffle thứ tự positions.
6. Parse video H.264 để lấy coefficients, CAVLC offsets, nC map, NAL length map, T1 override map.
7. Chạy CAVLC safety filter để lấy raw safe positions.
8. Nếu dùng locked operating point, nạp precomputed positions đã được validate.
9. Nếu không dùng locked positions, chạy FFmpeg validation tùy chọn, patchability pruning, và per-block limit.
10. Nhúng payload bits vào positions bằng T1 sign modification.
11. Reconstruct stego H.264.
12. Kiểm tra applied positions sau reconstruction.
13. Ghi `positions.json`, `meta.json`, và `manifest.json`.

16 Tại Sao Hệ Thống Hiện Mạnh Nhất Ở All-Intra H.264/CAVLC?

All-intra nghĩa là các frame chủ yếu được mã hóa độc lập, giống nhiều I-frame. Điều này hợp với hệ thống vì:

- sửa một frame ít gây cascade sang frame khác;
- residual blocks trong IDR/I-frame dễ kiểm soát hơn;
- CAVLC baseline có cấu trúc trailing-one phù hợp với sign-bit embedding;
- benchmark có thể khóa operating point ổn định hơn.

Với GOP lớn hoặc inter-coded video, P-frame/B-frame phụ thuộc dự đoán từ frame khác. Một thay đổi nhỏ có thể ảnh hưởng rộng hơn. Vì vậy inter-coded support là hướng cần phát triển thêm, không phải path mạnh nhất hiện tại.

17 Tóm Tắt Vai Trò Của Các Thành Phần

Thành phần	Vai trò
<code>src/embedder.py</code>	Public API. Tạo proof, pack payload, nạp analysis, chọn positions, nhúng, reconstruct, ghi sidecar.
<code>src/core/stego.py</code>	Safety filter và payload embed/extract logic. Đây là nơi tìm T1 positions và sửa coefficient.
<code>src/bitstream/cavlc.py</code>	CAVLC encoder/decoder. Dùng để hiểu và tái mã hóa residual blocks.
<code>src/bitstream/h264.py</code>	H.264 parser. Dùng để bóc NAL, slice, macroblock, block, coefficient.
<code>src/bitstream/bitstream_parser.py</code>	Opacity validation và reconstruction. Đây là phần đảm bảo thay đổi có thể sống trong bitstream thật.
<code>src/zk_proof.py</code>	Tạo, serialize, pack, unpack, và verify Groth16 proof.
<code>benchmark/_common.py</code>	Đo capacity layers và tái dùng analysis cache cho benchmark.

18 Điểm Cần Nhớ

Điểm tinh túy của hệ thống không phải là “đổi coefficient”. Điểm tinh túy là:

Chọn đúng coefficient mà sau khi đổi, H.264 bitstream vẫn vá được, video vẫn decode được, chất lượng vẫn đạt, và payload vẫn đọc lại được.

Vì thế, các thuật ngữ quan trọng nhất là:

- **CAVLC-safe**: vị trí không phá cấu trúc mã hóa coefficient.
- **Patchable**: block sau sửa có thể vá lại vào bitstream.
- **Reconstruct-aware**: sau reconstruction vẫn kiểm tra số bit sống thật.
- **Operating point**: cấu hình đã chạy thật và pass thật.
- **Capacity decomposition**: không đánh đồng raw candidates với capacity thật.

19 Kết Luận

Hệ thống xử lý video theo hướng rất cụ thể:

H.264 bitstream $\rightarrow$ IDR/CAVLC blocks $\rightarrow$ T1 sign positions $\rightarrow$ payload bits $\rightarrow$
patched stego H.264

Nó không giấu tin bằng cách sửa pixel. Nó giấu tin bằng cách sửa rất nhỏ dấu của một số residual coefficients đã được chọn lọc trong bitstream H.264. Chính những lớp lọc như patchability, reconstruction, quality guard, và operating capacity mới làm hệ thống khác với một phương pháp nhúng đơn giản.